

PROMPT:

Build a complete production-ready Model Context Protocol (MCP) Server using C# and .NET 8.

Do not modify any existing application code, business logic, frontend, backend structure, APIs, UI design, workflows, or project architecture unless absolutely required for MCP integration.

The project must use SQLite as the database and expose exactly three MCP tools through the MCP server.

Requirements:

- Use C# and .NET 8.
- Use SQLite for data storage.
- Follow MCP standards and best practices.
- Ensure compatibility with Claude Desktop.
- Use dependency injection and clean architecture principles.
- Implement proper logging, exception handling, validation, and configuration management.
- Support communication through standard MCP transport mechanisms.
- Maintain a professional folder structure and code organization.

You are an expert C# and .NET 8 architect. Build a complete, production-ready Model Context Protocol (MCP) Server project using C# and .NET 8.

Constraints:

- Do not modify existing application code, business logic, APIs, UI design, workflows, or architecture unless absolutely required for MCP integration.
- Use SQLite for database storage.
- Expose exactly three MCP tools: `list_tables`, `get_schema`, `run_select`.
- Ensure compatibility with Claude Desktop.
- Implement secure database access with safe read-only operations where applicable.
- Include proper logging, exception handling, validation, and configuration management.
- Support communication through standard MCP transport mechanisms.

Deliverables:

- Complete source code for every required file.
- Professional folder structure and project organization.
- All configuration files and application settings.

- Database initialization with sample data.
- README documentation with setup, installation, execution, architecture overview, assumptions, limitations, troubleshooting.
- Claude Desktop integration instructions and configuration examples.
- Architecture diagrams and project structure documentation.
- AI usage notes suitable for assignment submission.
- Setup, run, testing, and deployment instructions.

Expectations:

- Production-quality code ready for GitHub.
- End-to-end integration with Claude Desktop.
- Comprehensive error handling and structured responses.
- Maintain professional standards and best practices throughout.

OUTPUT:

The final output must be a fully functional, professional, GitHub-ready MCP Server project suitable for technical evaluation, demonstration, and end-to-end integration with Claude Desktop while preserving the existing application structure and functionality.